LAPORAN
WORKSHOP SISTEM INFORMASI
WEB SERVER
(TIF330805)
SEMESTER III



Acara 24

Nama:

Nila Agustin Prima Aurelia

NIM:

E41250666

Golongan:

A

PROGRAM STUDI DILUAR KAMPUS UTAMA (PSDKU)

TEKNIK INFORMATIKA KAB. NGANJUK

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI JEMBER

TAHUN 2026BAB

1. PENDAHULUAN

1.1 Latar Belakang

Pada praktikum sebelumnya, RESTful API data mahasiswa telah dibangun dengan PHP Native berpola MVC dan diuji menggunakan Postman. Pengujian tersebut hanya menampilkan data dalam format JSON, yang kurang mudah dibaca oleh pengguna umum. Agar data dapat dimanfaatkan secara nyata, API perlu dihubungkan dengan antarmuka (frontend) yang menampilkan data dalam bentuk yang mudah dipahami, misalnya tabel pada halaman web.

Integrasi ini dilakukan dengan JavaScript melalui Fetch API untuk mengambil data dari API secara asynchronous. Karena halaman frontend dan API dapat berasal dari origin yang berbeda, mekanisme Cross-Origin Resource Sharing (CORS) juga perlu dikonfigurasi pada sisi server. Praktikum ini membahas penerapan ketiga hal tersebut pada project si-akademik.

1.2 Tujuan Praktikum

1. Mengaktifkan header CORS pada Front Controller agar API dapat diakses dari origin lain.
2. Membuat view sederhana (HTML dan JavaScript) untuk menampilkan data mahasiswa dari API.
3. Menggunakan Fetch API dengan sintaks async/await untuk mengambil dan memproses response JSON.
4. Menampilkan indikator pemuatan data dan menangani kondisi data kosong pada tampilan.

BAB 2. DASAR TEORI

2.1 Fetch API sebagai Antarmuka Komunikasi HTTP

Fetch API menyediakan antarmuka JavaScript modern untuk melakukan permintaan HTTP dan memproses response, menggantikan XMLHttpRequest yang berbasis callback. Berdasarkan dokumentasi resmi MDN, pemanggilan fetch() mengembalikan sebuah Promise yang selesai (fulfilled) begitu server merespons dengan header, sehingga proses pembacaan body response (misalnya melalui method .json()) dilakukan pada tahap terpisah.

2.2 Asynchronous Request dengan Async/Await

Sintaks async/await memungkinkan kode asynchronous ditulis secara berurutan (sequential) sehingga lebih mudah dibaca dibandingkan rangkaian .then(). Penanganan kegagalan permintaan (misalnya jaringan terputus) dilakukan melalui blok try-catch di sekeliling pemanggilan await fetch(...).

2.3 Cross-Origin Resource Sharing (CORS)

CORS adalah mekanisme keamanan berbasis header HTTP yang mengatur apakah sebuah halaman web pada satu origin (domain, skema, atau port) diperbolehkan mengakses resource dari origin lain. Berdasarkan dokumentasi resmi MDN, browser akan menolak response dari server lain kecuali server tersebut secara eksplisit mengirimkan header seperti Access-Control-Allow-Origin yang mengizinkan permintaan tersebut.

BAB 3. HASIL PRAKTIKUM

3.1 Prosedur Kerja

View pertama dibangun pada folder public/views/, sementara header CORS ditambahkan satu kali pada Front Controller (public/index.php) agar berlaku untuk seluruh resource yang melewati Router.

Langkah 1. Mengaktifkan Header CORS pada Front Controller

Header CORS ditambahkan pada public/index.php sebelum Router dijalankan, sehingga berlaku otomatis untuk seluruh resource (mahasiswa, buku, login, dan register). Permintaan preflight (method OPTIONS) dijawab langsung dengan status 204 tanpa masuk ke Router.

File: public/index.php (diperbarui)

Acara24 > si-akademik > public > index.php

```php
<?php

require_once __DIR__ . '/../app/core/autoload.php';

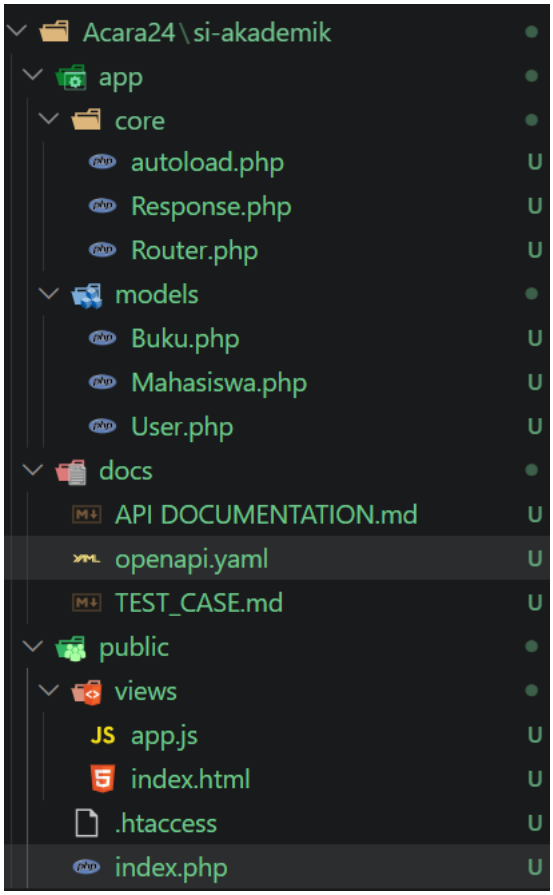
// Header CORS: berlaku untuk semua resource yang melewati Router
header('Access-Control-Allow-Origin: *');
header('Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS');
header('Access-Control-Allow-Headers: Content-Type, Authorization');

// Preflight request dari browser dijawab langsung, tanpa masuk Router
if (($_SERVER['REQUEST_METHOD'] ?? '') === 'OPTIONS') {
    http_response_code(204);
    exit;
}

(new Router())->dispatch();
```

Langkah 2. Membuat Folder View

Folder views dibuat di dalam folder public, berisi dua berkas yaitu index.html dan app.js. Karena berada di dalam public dan berkas fisiknya ada, aturan .htaccess tidak meneruskannya ke Router sehingga diakses sebagai berkas statis.



Langkah 3. Menulis Kode Fetch untuk Menampilkan Data

Kode pada app.js mengambil data dari endpoint GET /mahasiswa menggunakan fetch dengan async/await, lalu menampilkan setiap data ke dalam baris tabel. Alamat dasar API disesuaikan dengan nama folder project, yaitu si-akademik. Fungsi escapeHtml ditambahkan agar karakter khusus HTML pada data tidak dieksekusi sebagai tag.

File: public/views/app.js



```
Acara24 > si-akademik > public > views > JS app.js > ...
const BASE_URL = 'http://localhost/Web_Server/Acara24/si-akademik/public';

// Mencegah karakter HTML dari data dieksekusi sebagai tag
function escapeHtml(teks) {
    return String(teks ?? '').replace(/[&<>"']/g, (c) => ({
        '&': '&amp;', '<': '&lt;', '>': '&gt;', '"': '&quot;', "'": '&#39;'
    }[c]));
}

function tampilkanPesan(tbody, teks) {
    tbody.innerHTML = `<tr><td colspan="3">${teks}</td></tr>`;
}

async function muatDataMahasiswa() {
    const tbody = document.querySelector('#tabel-mahasiswa tbody');

    // Tugas Mandiri 1: indikator sebelum data berhasil diambil
    tampilkanPesan(tbody, 'Memuat data...');

    try {
        const response = await fetch(`${BASE_URL}/mahasiswa`);
        const hasil = await response.json();

        if (!response.ok || !hasil.success) {
            throw new Error(hasil.message || `HTTP ${response.status}`);
        }

        // Tugas Mandiri 2: penanganan data kosong
        if (!hasil.data || hasil.data.length === 0) {
            tampilkanPesan(tbody, 'Belum ada data');
            return;
        }

        tbody.innerHTML = '';
        hasil.data.forEach((mhs) => {
            tbody.innerHTML += `
                <tr>
                    <td>${escapeHtml(mhs.nim)}</td>
                    <td>${escapeHtml(mhs.nama)}</td>
                    <td>${escapeHtml(mhs.prodi)}</td>
                </tr>`;
        });
    } catch (error) {
        console.error('Gagal memuat data:', error);
        tampilkanPesan(tbody, 'Gagal memuat data');
    }
}

document.addEventListener('DOMContentLoaded', muatDataMahasiswa);
```

Langkah 4. Membuat View index.html

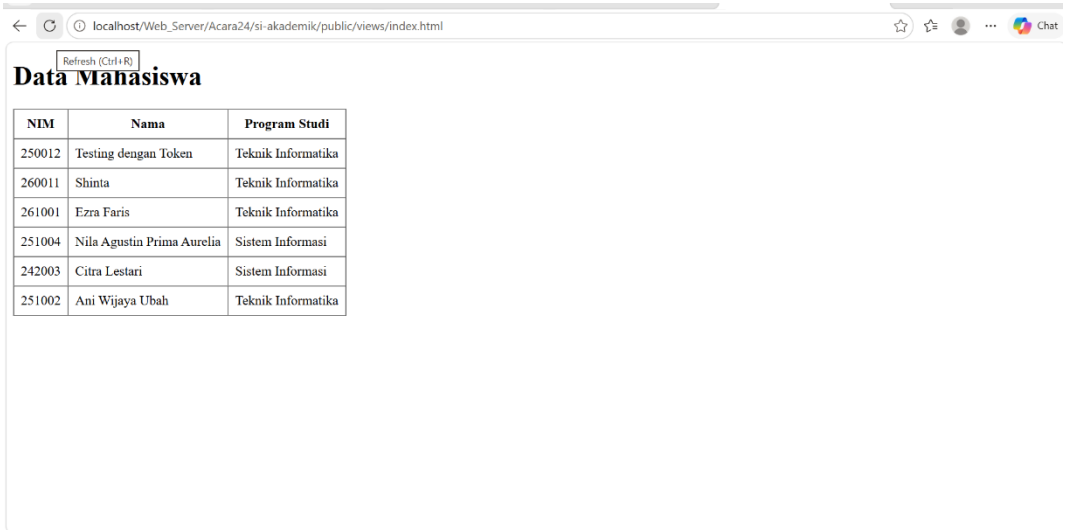
View berisi tabel dengan tiga kolom (NIM, Nama, Program Studi). Bagian tbody dibiarkan kosong karena akan diisi oleh JavaScript, kemudian berkas app.js dimuat di akhir body.

File: public/views/index.html

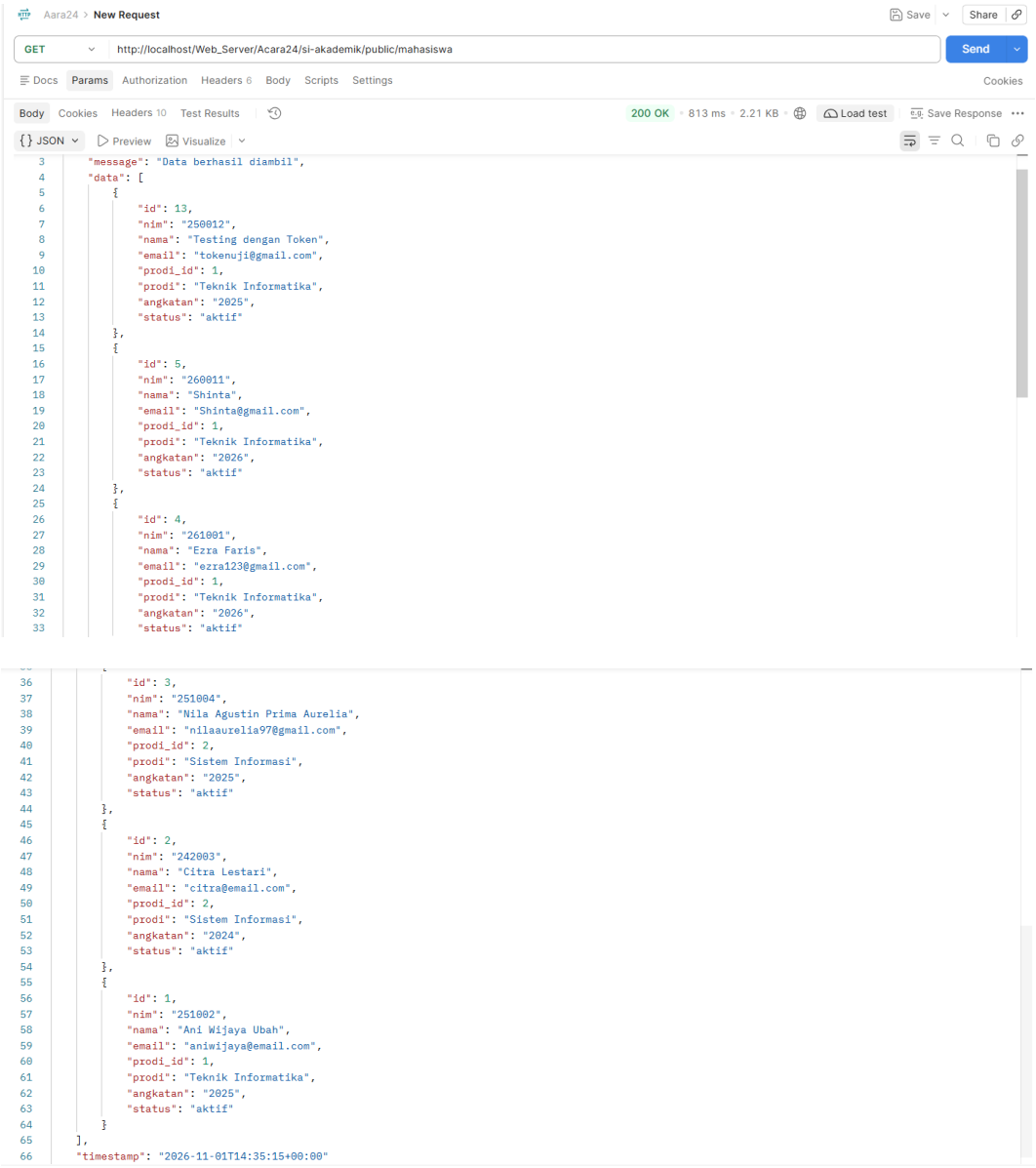
```
Acara24 > si-akademik > public > views > index.html > html
1  <!DOCTYPE html>
2  <html lang="id">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>Data Mahasiswa</title>
7  </head>
8  <body>
9      <h1>Data Mahasiswa</h1>
10     <table id="tabel-mahasiswa" border="1" cellpadding="8" cellspacing="0">
11         <thead>
12             <tr><th>NIM</th><th>Nama</th><th>Program Studi</th></tr>
13         </thead>
14         <tbody></tbody>
15     </table>
16     <script src="app.js"></script>
17 </body>
18 </html>
```

Langkah 5. Pengujian Tampilan View

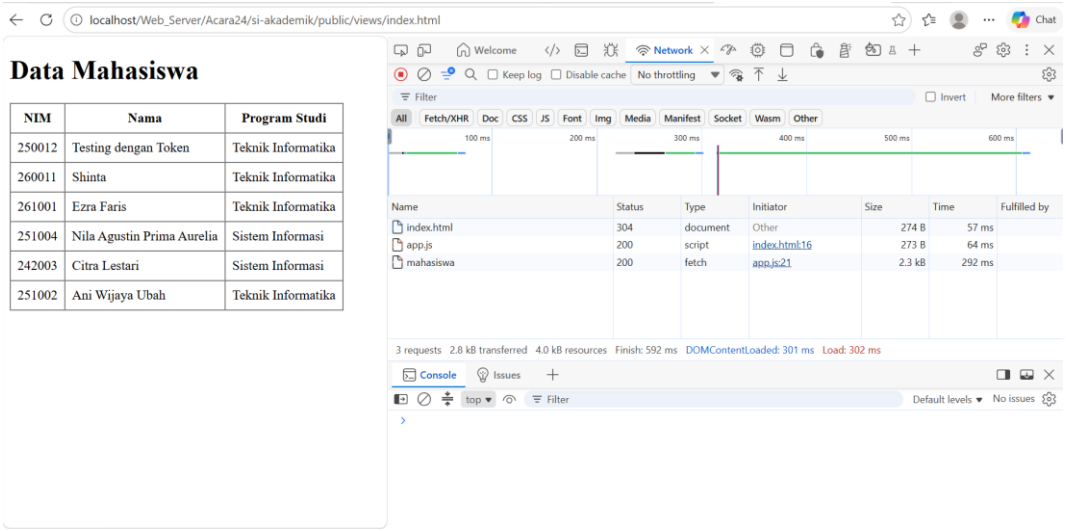
1. Layanan Apache dan MySQL pada Laragon dijalankan.
2. View dibuka melalui browser pada alamat http://localhostWeb_Server/Acara24/si-akademik/public/views/index.html.



3. Isi tabel HTML dibandingkan dengan hasil MahasiswaController::index() melalui endpoint GET http://localhost/Web_Server/Acara24/si-akademik/public/mahasiswa di Postman. Data pada tabel sesuai dengan data pada API, yaitu 6 data mahasiswa yang diurutkan dari id terbesar.



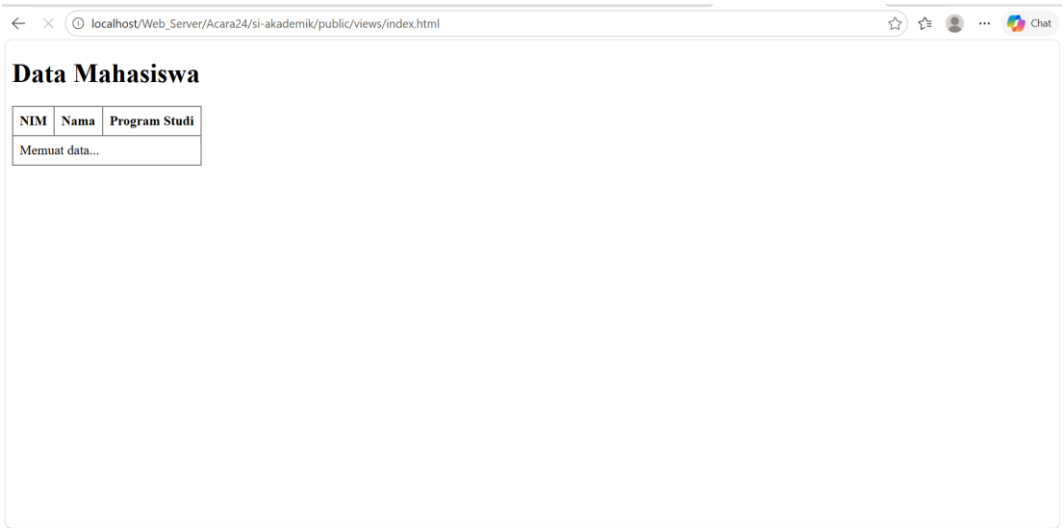
4. Developer Tools (Network dan Console) diperiksa. Request mahasiswa berstatus 200 OK, pada Response Headers terdapat Access-Control-Allow-Origin, dan pada Console tidak ada error CORS.



3.2 Tugas Praktik Mandiri

Tugas 1. Indikator teks "Memuat data..."

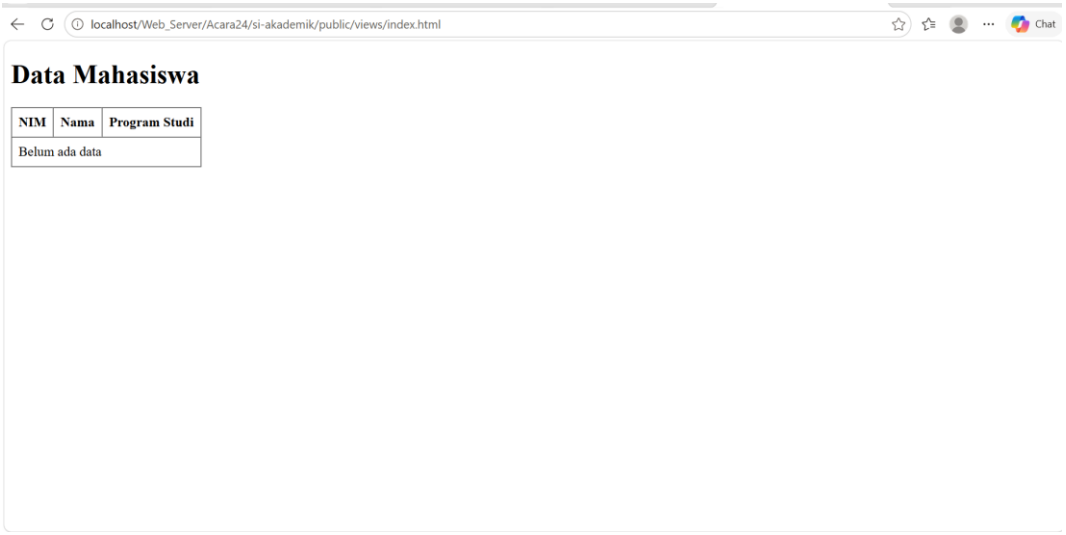
Sebelum fetch dijalankan, fungsi tampilkanPesan() mengisi tbody dengan satu baris berisi teks "Memuat data...". Teks ini otomatis tergantikan oleh data mahasiswa setelah response diterima. Karena pengujian dilakukan pada localhost yang sangat cepat, pengujian dilakukan dengan mengubah network throttling pada Developer Tools menjadi Slow 3G, sehingga indikator dapat terlihat sebelum data tampil.



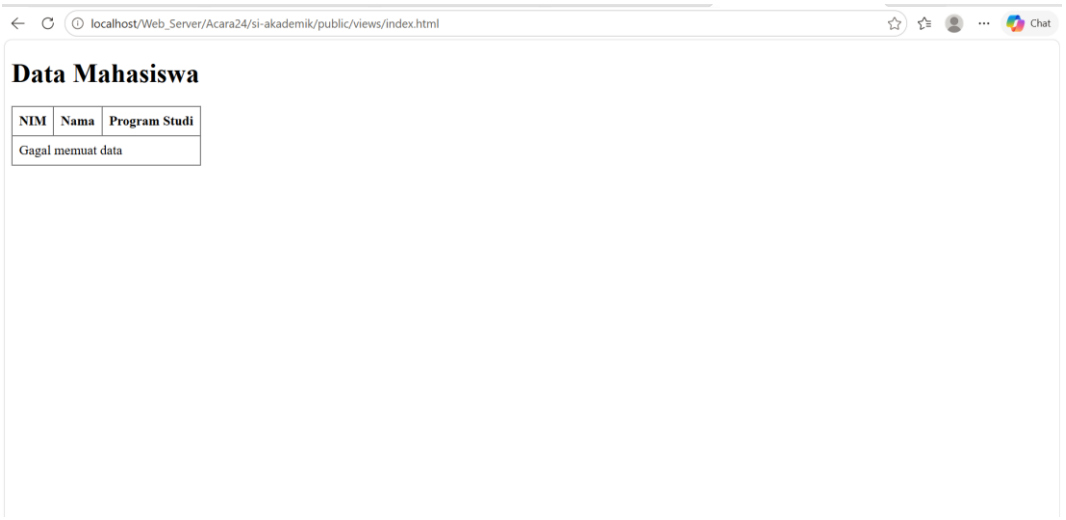
Tugas 2. Penanganan kondisi data kosong

Setelah response diterima dan dinyatakan berhasil, kode memeriksa apakah hasil.data kosong. Jika kosong, tabel menampilkan pesan "Belum ada data" dan proses dihentikan dengan return. Pengujian dilakukan dengan menambahkan

sementara baris hasil.data = []; setelah pemanggilan response.json(), kemudian baris tersebut dihapus kembali setelah pengujian selesai.



Sebagai tambahan, blok catch menampilkan pesan "Gagal memuat data" apabila request gagal, misalnya saat Apache dimatikan, sehingga pengguna tetap mendapat informasi pada halaman.



BAB 4. KESIMPULAN

Header CORS (Access-Control-Allow-Origin, Access-Control-Allow-Methods, dan Access-Control-Allow-Headers) cukup ditambahkan satu kali pada Front Controller public/index.php sehingga berlaku untuk seluruh resource. Permintaan preflight OPTIONS dijawab langsung dengan status 204.

Fetch API dengan async/await berhasil digunakan untuk mengambil data dari endpoint GET /mahasiswa dan menampilkannya pada tabel HTML. Data pada tampilan sesuai dengan data yang dikembalikan API.

Pemanfaatan try-catch membuat kegagalan request dapat ditangani, dan pemeriksaan response.ok serta hasil.success memastikan hanya response yang berhasil yang diproses. Indikator "Memuat data..." dan pesan "Belum ada data" membuat tampilan lebih informatif bagi pengguna pada kondisi data sedang dimuat maupun kosong.

Integrasi API dengan frontend membuktikan bahwa API yang telah dibangun dapat dikonsumsi oleh aplikasi klien, sehingga pemisahan antara backend dan frontend dapat diterapkan.